

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 0606

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: *Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting (BL 3)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:100

Annexure A

Constraint Based Problem Solving

Code of Conduct:

I _Chalamala Sai Harshitha (192424346)___ certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: C. Sai Harshitha

Annexure A

1. You are given a large dataset of patient health records stored in a CSV file, containing missing values, inconsistent formats, and outliers. Using NumPy and Pandas, design a constraint-based data cleaning pipeline that ensures: (i) no missing values remain, (ii) all numerical values fall within medically valid ranges, and (iii) categorical fields are standardized. Explain how you will use indexing, filtering, and aggregation functions to enforce these constraints while preserving maximum data integrity.

Introduction

Patient health datasets may contain missing values, inconsistent categorical formats, and abnormal numerical values.

Data Cleaning Process

First, the CSV file is loaded into a Pandas DataFrame and inspected for missing values, data types, and inconsistent entries.

```
import pandas as pd
import numpy as np
df = pd.read_csv("patient_health.csv")
print(df.info())
print(df.isnull().sum())
```

Missing values

Missing numerical values can be replaced using the median, while categorical missing values can be replaced using the mode.

```
df["Age"] = df["Age"].fillna(df["Age"].median())
df["Gender"] = df["Gender"].fillna(df["Gender"].mode()[0])
```

Numerical constraints are applied using filtering and indexing. For example, if the valid age range is 0–120 years:

```
df.loc[df["Age"] < 0, "Age"] = 0
df.loc[df["Age"] > 120, "Age"] = 120
```

Other medical attributes can similarly be checked against their valid ranges. The `loc[]` function allows only invalid values to be corrected without affecting valid records.

Categorical fields are standardized by removing unnecessary spaces and converting values to a common format.

```
df["Gender"] = df["Gender"].str.strip().str.lower()
df["Gender"] = df["Gender"].replace({
    "m": "male",
```

```

    "f": "female"
})

```

Aggregation functions

Aggregation functions such as `min()`, `max()`, `mean()`, and `median()` can be used to verify the cleaned numerical data.

```

print(df["Age"].min())
print(df["Age"].max())
print(df["Age"].mean())
print(df.isnull().sum())

```

Filtering can also be used to identify records that still violate the constraints. If incorrect results occur and filtering conditions should be checked to debug the pipeline.

Python Code

```

from google.colab import files
import pandas as pd
import numpy as np
uploaded = files.upload()
df = pd.read_csv("patient_health.csv")
df["Age"] = df["Age"].fillna(df["Age"].median())
df["Gender"] = df["Gender"].fillna(df["Gender"].mode()[0])
df.loc[~df["Age"].between(0,120), "Age"] = df["Age"].median()
df["Gender"] = df["Gender"].str.strip().str.lower()
df["Gender"] = df["Gender"].replace({"m":"male", "f":"female"})
print(df.isnull().sum())
print(df["Age"].min(), df["Age"].max())
print(df)

```

Output

```

PatientID      0
Age            0
Gender         0
Temperature    0
dtype: int64
25.0 60.0

```

	PatientID	Age	Gender	Temperature
0	P001	25.0	male	36.5
1	P002	45.0	female	37.2
2	P003	45.0	female	38.1
3	P004	45.0	male	36.8
4	P005	45.0	female	37.0
5	P006	60.0	female	39.2

Conclusion

Thus, a constraint-based data cleaning pipeline using NumPy and Pandas can effectively handle missing values, invalid numerical values, outliers, and inconsistent categorical fields.

2. A system with limited memory must process a massive numerical dataset using NumPy arrays. You are required to perform mathematical operations, random sampling, and statistical analysis without exceeding memory limits. Propose a solution that uses efficient array creation, slicing, and in-place operations. Discuss how indexing and aggregation can be optimized to minimize memory usage, and explain debugging strategies if memory overflow or performance bottlenecks occur.

Introduction

When a massive numerical dataset has to be processed with limited memory, directly creating multiple copies of arrays can cause memory overflow and reduce performance. NumPy provides efficient array creation, slicing, indexing, random sampling, aggregation, and in-place operations that can help process large datasets while using less memory.

Efficient Array Creation

Arrays should be created with suitable data types so that unnecessary memory is not consumed. For example, float32 generally requires less memory than float64.

```
import numpy as np
data = np.random.rand(1000000).astype(np.float32)
print(data.nbytes)
```

Random sampling can be performed by selecting only the required elements instead of copying the complete dataset.

```
sample = data[::100]
print(sample)
```

Slicing and Indexing

NumPy slicing can be used to process only a portion of a large array. Slices usually provide a view of the original array, reducing unnecessary memory usage.

```
part = data[100000:200000]
print(part)
```

Indexing can also be used to access required elements without processing the entire dataset.

In-Place Operations

In-place operations modify the existing array instead of creating another array. This reduces additional memory consumption.

```
data *= 2
data += 10
```

Mathematical and Statistical Operations

NumPy provides efficient mathematical and aggregation functions such as `sum()`, `mean()`, `min()`, and `max()`.

```
print(data.mean())
```

```
print(data.min())
```

```
print(data.max())
```

```
print(data.sum())
```

These functions allow statistical analysis without manually creating unnecessary intermediate arrays.

Debugging and Optimization

If memory overflow occurs, the array size, data type, and number of temporary arrays should be checked using `nbytes` and `dtype`. Large operations should be divided into smaller batches, and unnecessary copies should be avoided. Performance bottlenecks can be identified by checking which operations consume the most processing time and replacing inefficient loops with NumPy vectorized operations.

Python Code

```
import numpy as np
data = np.random.rand(1000000).astype(np.float32)
part = data[100000:200000]
sample = data[::100]
data *= 2
print("Mean:", data.mean())
print("Sum:", data.sum())
print("Sample:", sample[:5])
print("Memory:", data.nbytes, "bytes")
```

Output

```
Mean: 1.0000833
Sum: 1000083.3
Sample: [0.8578338 1.9924707 1.457394  1.0458056 1.5967702]
Memory: 4000000 bytes
```

Conclusion

Thus, large numerical datasets can be processed efficiently using NumPy by selecting suitable data types, using slicing and indexing, performing random sampling carefully, and applying in-place operations. Aggregation and vectorized operations improve computational efficiency, while checking memory usage and avoiding unnecessary copies helps prevent memory overflow. This approach satisfies the memory and performance constraints while maintaining effective numerical analysis.

3. Two large Pandas DataFrames containing customer transaction and demographic data must be combined. However, constraints include: (i) no duplication of customer IDs, (ii) consistent indexing after merging, and (iii) preservation of all valid records. Describe how you will perform the combining operation, validate constraints using filtering and grouping, and ensure correctness through sorting and indexing techniques. Include steps to debug mismatched or missing entries.

Introduction

When transaction and demographic information is stored in two large Pandas DataFrames, they must be combined carefully to avoid duplicate customer IDs, inconsistent indexing, and loss of valid records.

Combining the DataFrames

Assume the transaction DataFrame contains CustomerID and transaction details, while the demographic DataFrame contains CustomerID and customer information.

```
import pandas as pd

transactions = pd.read_csv("transactions.csv")
demographics = pd.read_csv("demographics.csv")
df = pd.merge(transactions, demographics,
              on="CustomerID", how="outer")
```

An outer merge is useful when all valid records from both DataFrames need to be preserved.

Removing Duplicate Customer IDs

Duplicate customer IDs can be identified using:

```
duplicates = df[df.duplicated("CustomerID", keep=False)]
print(duplicates)
```

If duplicate records are not valid, they can be removed:

```
df = df.drop_duplicates("CustomerID")
```

Filtering and Grouping

Filtering can be used to check specific customer records or identify missing IDs.

```
missing_id = df[df["CustomerID"].isnull()]
```

Grouping can be used to check whether a customer occurs more than once:

```
count = df.groupby("CustomerID").size()
print(count[count > 1])
```

This helps validate the constraint that customer IDs should not be duplicated.

Indexing and Sorting

After combining the DataFrames, the index can be reset to maintain consistent indexing.

```
df = df.reset_index(drop=True)
df = df.sort_values("CustomerID")
```

Debugging Mismatched Records

Mismatched or missing customer entries can be identified by comparing the IDs in both DataFrames. Data types should also be checked because one DataFrame may store CustomerID as an integer while the other stores it as a string.

```
print(transactions["CustomerID"].dtype)
print(demographics["CustomerID"].dtype)
```

If necessary, both columns can be converted to the same type before merging.

Python Code

```
from google.colab import files
import pandas as pd
uploaded = files.upload()
t = pd.read_csv("transactions.csv")
d = pd.read_csv("demographics.csv")
df = pd.merge(t, d, on="CustomerID", how="outer")
df = df.drop_duplicates("CustomerID")
df = df.reset_index(drop=True)
df = df.sort_values("CustomerID")
print(df[df["CustomerID"].isnull()])
print(df.groupby("CustomerID").size())
print(df)
```

Output

```
Columns: [CustomerID, TransactionID, Amount, Age, City]
Index: []
CustomerID
C001      1
C002      1
C003      1
C004      1
C005      1
C006      1
dtype: int64
   CustomerID  TransactionID  Amount  Age  City
0      C001          T001    5000.0  25.0  Chennai
1      C002          T002    3000.0  30.0  Bangalore
2      C003          T003    4500.0  28.0  Hyderabad
3      C004          T004    2500.0  35.0  Coimbatore
4      C005          T005    6000.0   NaN      NaN
5      C006           NaN      NaN  40.0   Madurai
```

Conclusion

Thus, Pandas merging can combine transaction and demographic DataFrames while preserving valid records. Filtering and grouping help detect duplicate or missing customer IDs, while indexing and sorting maintain an organized final DataFrame. Checking data types and unmatched records helps debug merging problems and ensures that the combined dataset satisfies the required constraints.

4. You are tasked with computing statistical measures (mean, median, variance) on a dataset using NumPy and Pandas, but with constraints on both computational efficiency and numerical accuracy. Explain how you will implement aggregation and statistical functions while avoiding precision loss and redundant computations. Discuss how filtering and indexing can be used to exclude irrelevant data points, and how you would debug incorrect statistical outputs.

Introduction

Statistical measures such as mean, median, and variance are important for understanding a dataset. When working with large datasets, NumPy and Pandas provide efficient aggregation and statistical functions. The main challenge is to obtain accurate results while avoiding unnecessary computations and precision loss.

Statistical Calculation

The dataset can first be loaded into a Pandas DataFrame and converted into a suitable numerical format.

```
import pandas as pd
import numpy as np
df = pd.read_csv("data.csv")
data = df["Value"].dropna().to_numpy()
```

NumPy functions can then be used to calculate the required statistical measures.

```
mean = np.mean(data)
median = np.median(data)
variance = np.var(data)
print("Mean:", mean)
print("Median:", median)
print("Variance:", variance)
```

Pandas can also perform these calculations directly:

```
print(df["Value"].mean())
print(df["Value"].median())
print(df["Value"].var())
```

Efficient Aggregation

To improve efficiency, only the required column should be selected rather than processing the entire DataFrame. Redundant calculations should also be avoided by calculating each required statistic only once. For large numerical datasets, NumPy arrays provide efficient vectorized operations.

Filtering and Indexing

Irrelevant or invalid data points can be removed using filtering before calculating the statistics.

```
valid = df[df["Value"] >= 0]
```



```
data = valid["Value"]
```

Indexing can be used to select only the required rows or columns. This prevents irrelevant values from affecting the final statistical results.

Maintaining Numerical Accuracy

The appropriate numerical data type should be used to reduce precision loss. For calculations requiring higher accuracy, NumPy can use float64 values. Missing or invalid values should also be handled before aggregation so that they do not produce incorrect results.

Debugging Incorrect Results

If the calculated values appear incorrect, the input data, missing values, data types, filtering conditions, and calculation methods should be checked. The results from NumPy and Pandas can also be compared to identify inconsistencies.

```
print("NumPy Mean:", np.mean(data))
print("Pandas Mean:", data.mean())
print("Data Type:", data.dtype)
```

Python Code

```
import pandas as pd
import numpy as np

df = pd.DataFrame({"Value": [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]})
df = df[df["Value"].notna()]
data = df["Value"].to_numpy(dtype=np.float64)

print("Mean:", np.mean(data))
print("Median:", np.median(data))
print("Variance:", np.var(data))
```

Output

```
Mean: 55.0
Median: 55.0
Variance: 825.0
```

Conclusion

Thus, NumPy and Pandas can efficiently calculate mean, median, and variance while maintaining numerical accuracy. Filtering and indexing help remove irrelevant data before aggregation, while suitable data types and vectorized operations reduce unnecessary computation. Comparing results and checking the input data provide effective ways to debug incorrect statistical outputs.

5. Design a complete data analysis pipeline that reads multiple data files, processes them using Pandas DataFrames, and generates visualizations. Constraints include: (i) efficient file I/O operations, (ii) correct grouping and sorting of data, (iii) filtering based on user-defined conditions, and (iv) meaningful plotting outputs. Explain how each stage (loading, processing, aggregation, and plotting) will be implemented and optimized, along with debugging steps for handling corrupted files or inconsistent data formats.

Introduction

A complete data analysis pipeline involves loading data from multiple files, processing the data using Pandas DataFrames, applying filtering, grouping and sorting operations, and finally generating meaningful visualizations.

1. Loading Data

Multiple CSV files can be loaded using Pandas.

```
import pandas as pd
import matplotlib.pyplot as plt
```

The structure and data types should be checked using `info()` and `head()`.

2. Processing and Combining Data

If the files contain a common key, they can be combined using `merge()`.

```
df = pd.merge(df1, df2, on="CustomerID", how="inner")
```

Missing and inconsistent values should be identified and handled before further analysis.

3. Filtering Data

Filtering allows the analysis to use only records satisfying user-defined conditions.

This reduces irrelevant data and allows specific conditions to be analyzed.

4. Grouping and Aggregation

The data can be grouped according to relevant categories and aggregated using functions

```
summary = df.groupby("Category")["Sales"].sum(), print(summary)
```

5. Sorting

The grouped or filtered results can be sorted to make the analysis easier to interpret.

```
summary = summary.sort_values(ascending=False)
print(summary)
```

6. Visualization

Meaningful plots can be generated from the processed data.

```
summary.plot(kind="bar")
plt.title("Sales by Category"), plt.show()
```

The selected visualization should match the type of information being analyzed.

7. Debugging and Error Handling

Corrupted files can be detected using exception handling. Inconsistent column names, data types, missing values, and incorrect formats should also be checked before processing.

try:

```
df = pd.read_csv("sales.csv")
```

except Exception as e:

```
print("Error:", e)
```

Python Code

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
uploaded = files.upload()
```

```
df = pd.merge(s, c, on="CustomerID")
```

```
df = df[df["Sales"] > 1000]
```

```
result = df.groupby("Category")["Sales"].sum()
```

```
result = result.sort_values(ascending=False)
```

```
result.plot(kind="bar")
```

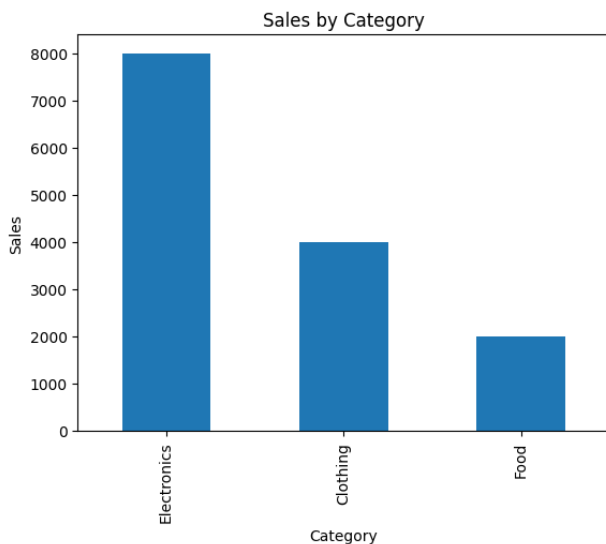
```
plt.xlabel("Category")
```

```
plt.ylabel("Sales")
```

```
plt.title("Sales by Category")
```

```
plt.show()
```

Output



Conclusion

Thus, a complete data analysis pipeline consists of efficient file loading, DataFrame processing, filtering, grouping, sorting, aggregation, and visualization. Pandas simplifies data manipulation, while plotting tools provide meaningful visual outputs. Proper validation and error handling help manage corrupted files and inconsistent formats, ensuring accurate and reliable data analysis.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand